

0.12.3. La fonction attendre() — attendre des secondes

#include <attendre> : attendre(2) arrête tout le programme deux secondes, puis il continue.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `attendre(secondes)` **arrête tout** le programme ce nombre de secondes, puis il continue à la ligne suivante. Simple, mais pendant ce temps rien d'autre ne se passe : la manette n'est pas lue. Pour attendre sans tout arrêter, il y a `chaque()`.

La ligne à écrire : `#include <attendre>`.

Son argument : le nombre de secondes, jusqu'à 255.

Ce qu'elle coûte : une vingtaine d'octets.

Essaie : ajoute une troisième ligne, TROIS, après un autre `attendre(1);`.

On retrouve attendre dans 3 leçons

[0.13](#), [0.13.1](#), [0.13.2](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **332 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	34 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
AttendreSecondes	17 o	← attendre() — attend des secondes entières
CopierTuiles	58 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	80 o	← CopierTuiles — 5 lettres sur 43
"UN"	2 o	← texte() — un texte gravé
"DEUX"	4 o	← texte() — un texte gravé

Lettres gravées : DENUX

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	x 2	22 o sur place, dans main()	+ EcrireTexte (10 o)
attendre()	x 1	5 o sur place, dans main()	+ AttendreSecondes (17 o)
image()	x 1	3 o sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

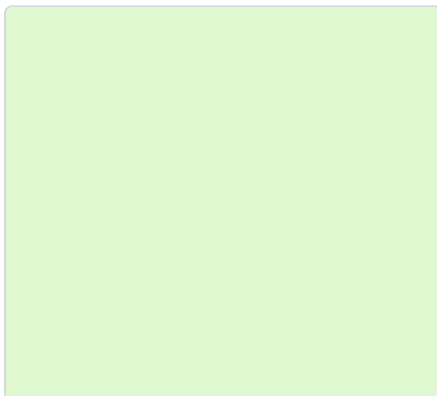
```
// ---- #include <attendre> : attendre des secondes ----  
  
#include <attendre>    // attendre() : tout s'arrête un moment  
#include <texte>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

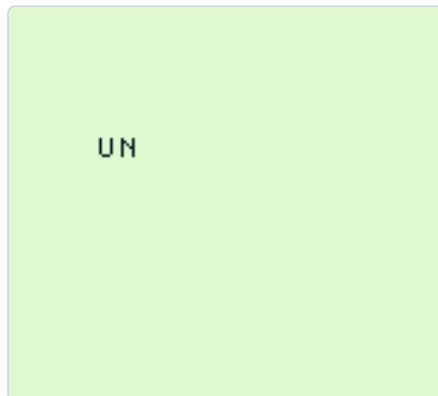
```
int main() {  
    texte(4, 6, "UN");  
    attendre(2);           // deux secondes : rien ne bouge  
    texte(4, 8, "DEUX");   // puis la suite  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout de trois secondes — image 180

4. Ce qu'on doit voir

UN, puis DEUX deux secondes plus tard.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ UN est là tout de suite
- ✓ DEUX n'est pas encore là au départ
- ✓ deux secondes plus tard, DEUX est là

